Advanced Terraform on Azure
Lab 5 – Modularization, Contracts and Validations

Contents

Expected Duration

This lab should take 45-55 minutes to complete

Teaching Points

At the end of Module 4, we had a hardened, well-structured monolithic Terraform configuration. It deployed a Resource Group, VNet, subnets, an NSG, and NSG rules using dynamic maps and allow-groups. The code enforces naming, cleans inputs, validates CIDRs, and uses remote state files across dev\test\prod environments. Module 5 now evolves this monolith into two clean, reusable modules. The goal is to restructure - not redesign - while producing zero infrastructure changes.

The Problem We Are Solving

The Module 4 configuration is correct but not modular. All logic lives at the root. It cannot be reused or easily extended. Real-world Terraform uses modules for separation of concerns, flexibility, and reusability. We will now break the monolith into modules without changing deployed resources. When you move resources from the root into modules, their terraform addresses change. Terraform interprets this as deletion and recreation. This churn is expected behaviour. We can use `moved` code blocks to tell Terraform that resources have been moved, thus preventing churn.

Key principles:

- Root modules should behave like thin orchestrators.
- Module contracts are defined by inputs (variables), validation, and outputs.
- Validation belongs both at the root (external inputs) and module level (assumed invariants).
- Refactors should aim for zero-diff plans – behaviour must not change when moving to modules.

Lab Rules

- Work only inside the **lab5\guided** folder for the main configuration.
- Always specify **-var-file=<env>.tfvars** when deploying to dev\test\prod.
- Aim for a zero-diff plan after modularization – investigate any unexpected changes.

Prerequisites

This lab assumes the presence of a remote state backend, as created in Lab 4. If this does not exist due to lab restarts etc. then run the following script in a VSC terminal session to recreate it …

```
C:\qatfadvaz\artifacts\fixes\bootstrap-backend.ps1
```

Pause for thought

Before you begin, think about the following:

- Where are the natural boundaries for modules in your current configuration?
- Which resources belong to “network” concerns, and which belong to “security” concerns?

Phase 1 – Baseline: Review the Monolith and Capture a Plan

This phase establishes a behavioural baseline. The goal is not to change anything yet, but to capture what "correct" looks like before refactoring. Every later plan will be judged against this baseline. We will redeploy the Module 4 monolith using the new backend. This stabilizes the world before modularization.

1. In **lab5\guided**, update **providers.tf** with your **subscription id** and your remote storage account **suffix**.
2. Run ...

   ```
   cd c:\qatfadvaz\lab5\guided
   terraform init -reconfigure -backend-config="key=dev.tfstate"
   terraform apply --var-file=dev.tfvars --auto-approve
   ```

   This is now your baseline deployment before modularization.

Verification

3. Confirm deployment of the twelve expected resources: the resource group, the Vnet with three subnets, the NSG with three rules, and the association of the three subnets to the NSG. All of the deployment is tracked in the **dev.tfstate** statefile.

Takeaway

You now have a working baseline plan against which you can compare the modularized configuration. Any behavioural change introduced by refactoring will be visible as a diff from this plan.

Refactoring Terraform should always start with a known-good plan. If you do not know what "no change" looks like, you cannot detect accidental change later.

Phase 2 – Introduce modules\security (NSG and Rules)

In this phase, we intentionally break the configuration to rebuild it correctly. Terraform will show errors as we progress through this phase. This is expected.

Steps you will perform:

- Create a security module
- Move NSG + rule resources from root into this new module
- Replace RG references with variables
- Add variables, locals and outputs in the security module
- Call module from root
- Expect seven destroys \ seven creates when planning

Moving resources into a module changes their Terraform addresses. Terraform will interpret this as “old resources removed, new resources added”. This is expected behaviour, not a mistake.

4. **Move** (not copy) two resource blocks out of **lab5\guided\main.tf** and into **lab5\guided\modules\security\main.tf** …

   azurerm_network_security_group.nsg
   azurerm_network_security_rule.rule

5. Expect six errors. Close and re-open VSC if six error notifications do not appear after the move …

```
resource "azurerm_network_security_group" "nsg" {
  name                = "${local.prefix}-nsg"
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
  tags                = local.base_tags
}

resource "azurerm_network_security_rule" "rule" {
  for_each = local.nsg_rules_normalised

  [
    for g in each.value.allow_groups :
    lookup(local.allow_groups_clean, g, [])
  ]

  [
    for g in each.value.allow_groups :
    lookup(local.allow_groups_clean, g, [])
  ]
```

6. In **lab5\guided\modules\security\main.tf**, replace …

   The two instances of **azurerm_resource_group.rg.name** with **var.resource_group_name**

   local.prefix with **var.prefix**

   local.base_tags with **var.base_tags**

```
resource "azurerm_network_security_group" "nsg" {
  name                = "${var.prefix}-nsg"
  location            = var.location
  resource_group_name = var.resource_group_name
  tags                = var.base_tags
}

resource "azurerm_network_security_rule" "rule" {
  for_each = local.nsg_rules_normalised

  destination_address_prefix  = "*"
  resource_group_name         = var.resource_group_name
  network_security_group_name = azurerm_network_security_g

  [
    for g in each.value.allow_groups :
    lookup(local.allow_groups_clean, g, [])
  ]

  [
    for g in each.value.allow_groups :
    lookup(local.allow_groups_clean, g, [])
  ]
```

Pause for thought… Why are these changes necessary ?

7. The error count should have risen to eight. No variables or locals been declared in the module yet, hence the error messages. We will now address this.
8. Add the following to **lab5\guided\modules\security\variables.tf**;

```
variable "location" {}
variable "prefix" {}
variable "resource_group_name" {}
variable "base_tags" {}
variable "allow_groups" {}
variable "nsg_rules" {}
```

9. This will reduce the error count to three, the undeclared locals …

```
resource "azurerm_network_security_rule" "rule" {
  for_each = local.nsg_rules_normalised

[
  for g in each.value.allow_groups :
  lookup(local.allow_groups_clean, g, [])
]

[
  for g in each.value.allow_groups :
  lookup(local.allow_groups_clean, g, [])
]
```

10. **Move** the following locals from **lab5\guided\main\local.tf** to **lab5\guided\modules\security.locals.tf** ...

    allow_groups_clean
    nsg_rules_clean
    nsg_rules_normalised

11. Add the following to **lab5\guided\modules\security\outputs.tf** ...

```
output "nsg_id" {
 value     = azurerm_network_security_group.nsg.id
}
```

12. In the **lab5\guided\main.tf**, replace the removed NSG resources with a module block:

```
module "security" {
 source = "./modules/security"

 prefix = local.prefix
 location = var.location
 resource_group_name  = azurerm_resource_group.rg.name
 base_tags  = local.base_tags
 nsg_rules = var.nsg_rules
 allow_groups = var.allow_groups
}
```

13. Still in **main.tf**, locate the **security group association** resource block, and replace **azurerm_network_security_group.nsg.id** with ...

    module.security.nsg_id

    Why? At present, the networking resources are still defined in the root module. The id of the NSG, needed for the networking subnet-to-NSG association, is no longer known at root. The security module must therefore output information to the root module where it can be referenced.

14. Run ...

```
cd c:\qatfadvaz\lab5\guided
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
```

Verification

Confirm that the plan proposes seven replacements. These are four resources that have been moved into the security module (The NSG and its three rules), and reconfiguration of three root-level rule-to-subnet associations. This churn is not desirable behaviour.

Takeaway

You have now extracted security concerns into modules\security but have introduced potential configuration and state churn. Modules do not inherit context. Every value a module needs must be passed explicitly as an input, even if that value previously existed in the root module.

Phase 3 - Refactor Without Destruction Using moved Blocks

This phase teaches you how to use Terraform's moved block feature to avoid destroying real infrastructure when refactoring resources into modules. Terraform tracks resources by address, so modularization changes addresses and may trigger destructive behaviour unless a moved block is provided.

A moved block performs its migration exactly once. After Terraform has updated the state for all environments, the block becomes a no-op. However, in real projects you should keep the moved block in your Terraform code until you are certain that all environments—and all team members—have applied the updated configuration.

If a moved block is removed too early, Terraform will assume the old address was dropped and may try to destroy and recreate existing infrastructure.

In production, moved blocks typically stay in the codebase for weeks or months and are removed only during a deliberate, controlled cleanup refactor once all state files have been fully migrated.

Implementation

15. A **moved.tf** file has been provided, detailing the resource moves, but it is currently inactive. Comment out lines **2** and **19** to activate the move of resources into the security module.

16. Run **terraform plan --var-file=dev.tfvars**

Verification

This should result in a zero-difference plan.

Takeaway

Using ‘moved’ allows modularization without churn. Any time you refactor Terraform structure without intending to change infrastructure, expect to pair that refactor with explicit state migration instructions.

Phase 4 – Introduce modules\network (VNet and Subnets)

Try in yourself (Optional)

17. Based on the processes outlined in phases 2 and 3 above, move the vnet and subnet resources currently in the root module into a dedicated 'network' module with zero-difference, uncommenting lines 22 and 51 of moved.tf when appropriate.

Step-by-step

This phase deliberately repeats the previous pattern with different resources. The goal is not novelty, but reinforcement: modularisation follows the same principles regardless of resource type.

18. **Move** three network resource blocks out of **main.tf** and into **modules\network\main.tf** …

    azurerm_virtual_network.vnet
    azurerm_subnet.subnet
    azurerm_subnet_network_security_group_association.subnet

19. Expect five errors. Close and re-open VSC if no error notifications appear after the move.

20. In **network\main.tf**; replace …

    Two instances of **azurerm_resource_group.rg.name** with **var.resource_group_name**

    azurerm_resource_group.rg.location with **var.location**

    Two instances of **local.prefix** with **var.prefix**

    local.base_tags with **var.base_tags**

    module.security.nsg_id with **var.nsg_id**

21. **Move** the following local from **guided\main\local.tf** to **guided\modules\network.locals.tf** …

    subnet_cidrs_clean

22. Create the following in **modules\network\variables.tf** …

```
variable "location" {}
variable "prefix" {}
variable "resource_group_name" {}
variable "base_tags" {}
variable "nsg_id" {}
variable "vnet_address_space" {}
variable "subnet_cidrs" {}
variable "approved_vnet_cidrs" {}
```

23. No error reports should remain

24. In the **guided\main.tf**, replace the removed network resources with a module block:

```
module "networking" {
 source          = "./modules/network"

 location = var.location
 resource_group_name = azurerm_resource_group.rg.name
 base_tags = local.base_tags
 prefix = local.prefix
 vnet_address_space = var.vnet_address_space
 subnet_cidrs = var.subnet_cidrs
 nsg_id = module.security.nsg_id
 approved_vnet_cidrs = var.approved_vnet_cidrs
}
```

Design principle: Modules should expose only what other modules need. Outputs define a module's public interface in the same way return values define a function's interface.

25. Plan the changes…

```
cd c:\qatfadvaz\lab5\guided
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
```

26. Seven destroy\adds are proposed (The Vnet, the three subnets and the three associations). More possible churn!

27. Comment out lines **22** and **51** of the provided **moved.tf**

28. Run; **terraform plan --var-file=dev.tfvars**

29. This should show a zero-diff move to a modularization of networking

Phase 5 – Contracts and Validation

What changes here? Until now, validation lived mostly at the root. From this point onward, modules become responsible for defending their own assumptions. This is the transition from "working code" to "robust code".

30. Open and review **modules\security\variables.tf** and **modules\network\variables.tf**.

31. The variables needed for the modules have been defined, but there are untyped and have no validation checks being performed. At present the checks are being performed at the root module level. Whilst this works in principle, in practice the modules should

‘stand-alone’ with the root module performing minimal high-level verification of data that will be passed into the module.

Implementation

32. From **guided\variables.tf**, copy the following variable definition blocks into **network\variables.tf**;

    vnet_address_space
    subnet_cidrs
    approved_vnet_cidrs

33. In **network\variables.tf**, remove the initially defined {} variables matching those now fully defined in the previous step.

34. From **guided\variables.tf**, copy the following variable definition blocks into **security\variables.tf**;

    allowed_groups
    nsg_rules

35. In **security\variables.tf**, remove the initially defined {} variables matching those now fully defined in the previous step.

36. In **guided\variables.tf**; remove the validation portions of the following variable definitions;

    allowed_groups
    nsg_rules
    vnet_address_space
    subnet_cidrs

37. In **guided\variables.tf**; remove the following variable definition;

    approved_vnet_cidrs

38. In **guided\locals.tf**; remove the following definitions …

    allow_groups_clean
    nsg_rules_clean
    nsg_rules_normalised
    subnet_cidrs_clean

39. In the **guided\main.tf**, remove the **approved_vnet_cidrs** entry from the networking module block, as this is now a purely module-level parameter …

```
module "networking" {
 source          = ".\modules\network"
 ....
```

```
  approved_vnet_cidrs = var.approved_vnet_cidrs
}
```

Verification

40. Confirm these changes do not introduce churn;

    terraform plan --var-file=dev.tfvars

41. No changes should be proposed

42. Introduce errors at the root module to verify that they are captured and we fail fast and early. Do this by altering values in **guided\dev.tfvars** and find the impact by running;

    terraform plan --var-file=dev.tfvars

43. Roll-back any changes made

Takeaway

You have strengthened the contracts for your modules. Validation now protects against misconfiguration at the module boundary, complementing root-level validation on raw inputs.

Phase 6 – Module level parameterization

The network and security modules currently inherit parameter values from the root module. Alongside these root-level parameters, there will often be a need for module-specific parameters.

In this phase we will take the root-level base tags and merge them with tags specific to the modules. No module-level tags will be added in this phase; they will be placeholders ahead of future steps.

44. In **modules\network\locals.tf**, add the following ..

```
mod_tags = merge(
 var.base_tags,
 {}
)
```

45. Repeat this in **modules\security\locals.tf**

46. In **modules\network\main.tf**, update the tagging of the vnet from **var.base_tags** to **local.mod_tags**

47. Repeat this for the tagging of the NSG in **lab5\guided\modules\security\main.tf**

48. Confirm these changes do not introduce any churn;

    terraform plan --var-file=dev.tfvars

Phase 7 – Zero-Diff Refactor Across Environments

49. Plan and Apply workflows for all three environments, using the same backend keys as previously …

    terraform init -reconfigure -backend-config="key=dev.tfstate"
    terraform plan --var-file=dev.tfvars
    terraform apply --var-file=dev.tfvars --auto-approve

    terraform init -reconfigure -backend-config="key=test.tfstate"
    terraform plan --var-file=test.tfvars
    terraform apply --var-file=test.tfvars --auto-approve

    terraform init -reconfigure -backend-config="key=prod.tfstate"
    terraform plan --var-file=prod.tfvars
    terraform apply --var-file=prod.tfvars --auto-approve

50. The dev environment deployment should be a no-diff as it is already deployed, and no module-level tagging was performed. The test and prod environments are fresh deployments and should succeed. The moved.tf code should be a no-op.
51. Use the portal to confirm all three environments deployments now exist simultaneously, tracked by a unique backend statefile in a shared storage account.
52. Given that, in this lab, there are no other team members who may be relying on the 'moved' blocks, you can safely delete **moved.tf** as it is now a pure no-ops across all environments.

You have confirmed that the modularization is behaviour-preserving across dev, test, and prod. Zero-diff refactors provide confidence that structural improvements have not changed the actual infrastructure.

Phase 8 – Clean Up

53. Ensure you are in **lab5\guided** and run;

    terraform init -reconfigure -backend-config="key=dev.tfstate"
    terraform destroy --var-file=dev.tfvars --auto-approve

    terraform init -reconfigure -backend-config="key=test.tfstate"
    terraform destroy --var-file=test.tfvars --auto-approve

    terraform init -reconfigure -backend-config="key=prod.tfstate"
    terraform destroy --var-file=prod.tfvars --auto-approve

Verification

54. Use the Azure portal to confirm that all lab resources have been destroyed except for the remote backend.

Takeaway

You have completed the modularization lab. You now have a repeatable pattern for turning a flat configuration into a set of reusable, validated modules while preserving behaviour across environments. We will next look at invoking this modular approach as the starting pattern rather than by a refactoring process as was performed here. We will also discuss module version and remote hosting of modules.

Copyright